AI - Lab

1. PyTorch Introduction



Hochschule Karlsruhe
Technik und Wirtschaft
UNIVERSITY OF APPLIED SCIENCES

Dr. Fabian Sigloch

WS 26/27

Motivation

Hochschule Karlsruhe University of Applied Sciences HKA

- PyTorch is nowadays probably the most popular Deep Learning framework. (It surpassed TensorFlow in this regard).
- It is based on easy concepts that allow to build complex model architectures.
- We will introduce the concepts of PyTorch by seeing how it can replace an implementation of a Neural Network that is based on `numpy`.

What is Numpy?

Hochschule Karlsruhe University of Applied Sciences HKA

- Numpy stands for “Numerical Python”.
- Numpy is a library that contains functions for fast mathematical computation with Python. For the speed-up, it relies on C-language bindings.
- Its concept is similar to Matlab, i.e. working mainly with arrays and matrices.
- As a machine learning person you should know it, since many important libraries (e.g. Sklearn, Pandas, and more) are using it under the hood.
- We will shortly introduce it, to see how PyTorch extends on it.
- Check the tutorial here.

Neural Networks in Numpy

Hochschule Karlsruhe University of Applied Sciences HKA

As an introduction exercise, we look at what it takes to build the following neural-network architecture from scratch in Python using Numpy (the original source can be found here).

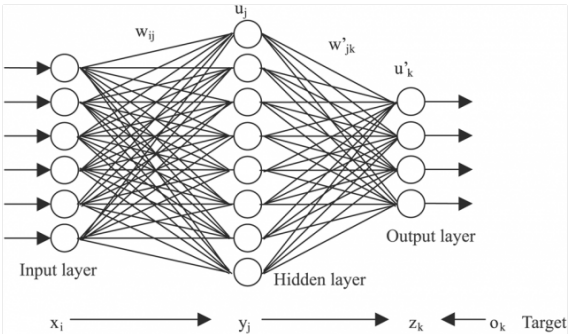


Image:

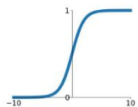
https://www.extremetech.com/extreme/215170-artificial-neural-networks-are-changing-the-world-what-are-they

Neural Network in Numpy

- Other than in the image, we use the following dimensions:
  - Input layer: 1000 nodes
  - Hidden layer: 100 nodes
  - Output layer: 10 nodes
- We use "ReLU" as activation function in the hidden layer.
- We use Euclidean error as loss (warning: this is not very typical).
- We use random input and random output data as training data.
- Weight parameters are learned via back propagation.

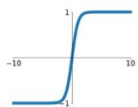
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



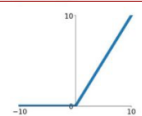
tanh

$\tanh(x)$



ReLU

$\max(0, x)$



Neural Network in Numpy

Hochschule Karlsruhe
University of
Applied Sciences
HKA

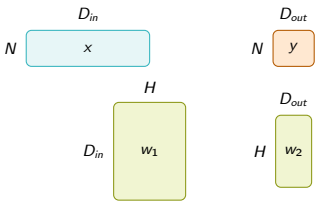
Data and parameter dimensions

```
import numpy as np

# N: batch size; D_in: input dimension
# H: hidden size; D_out: output size
N, D_in, H, D_out = 64, 1000, 100, 10

# Create random input and output data
x = np.random.randn(N, D_in) # input
y = np.random.randn(N, D_out) # target

# Randomly initialize weights
w1 = np.random.randn(D_in, H) # first weights
w2 = np.random.randn(H, D_out) # second weights
```



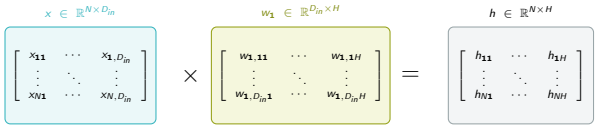
Neural Network in Numpy

Hochschule Karlsruhe University of Applied Sciences HKA

Forward pass, step 1: project the input into the hidden space

Training loop

```
learning_rate = 1e-6
for t in range(500):
  # Forward pass
    h = x.dot(w1)
  h_relu = np.maximum(h, 0)
  y_pred = h_relu.dot(w2)
  # Compute loss
  loss = np.square(y_pred - y).sum()
  if t % 50 == 9:
    print(t, loss)
```



- The two inner dimensions must match: $D_{in} = 1000$.
- The batch dimension $N = 64$ is retained: all samples are processed at once.

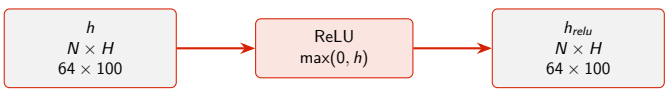
Neural Network in Numpy

Forward pass, step 2: add non-linearity

Hochschule Karlsruhe University of Applied Sciences HKA

Training loop

```
learning_rate = 1e-6
for t in range(500):
  # Forward pass
  h = x.dot(w1)
  h_relu = np.maximum(h, 0)
  y_pred = h_relu.dot(w2)
  # Compute loss
  loss = np.square(y_pred - y).sum()
  if t % 50 == 9:
    print(t, loss)
```



Without an activation function, two linear layers would still be equivalent to one linear layer.

Neural Network in Numpy

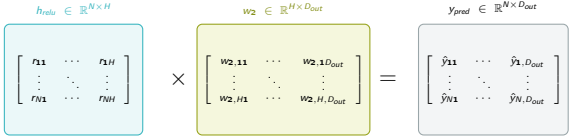
Forward pass, step 3: calculate class scores

Hochschule Karlsruhe
University of
Applied Sciences
HKA

Training loop

```
learning_rate = 1e-6
for t in range(500):
  # Forward pass
  h = x.dot(w1)
  h_relu = np.maximum(h, 0)
  y_pred = h_relu.dot(w2)

  # Compute loss
  loss = np.square(y_pred - y).sum()
  if t % 50 == 9:
    print(t, loss)
```



One row of y_{pred} contains ten output values for one sample.

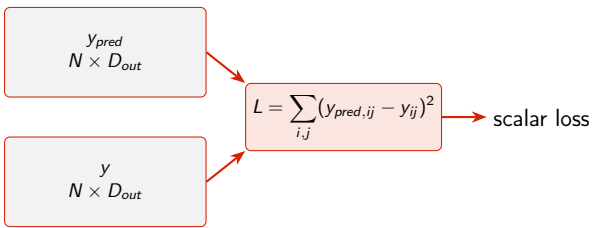
Neural Network in Numpy

Compare prediction and target

Hochschule Karlsruhe
University of
Applied Sciences
HKA

Training loop

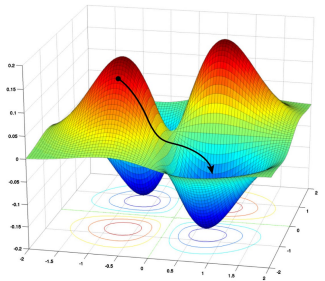
```
learning_rate = 1e-6
for t in range(500):
  # Forward pass
  h = x.dot(w1)
  h_relu = np.maximum(h, 0)
  y_pred = h_relu.dot(w2)
  # Compute loss
    loss = np.square(y_pred - y).sum()
  if t % 50 == 9:
    print(t, loss)
```

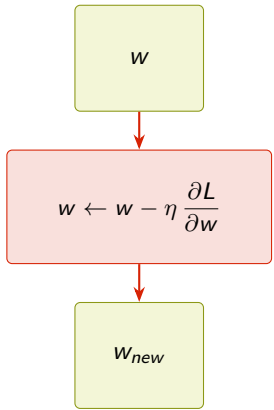


- y_{pred} and y must have exactly the same shape: $(64, 10)$.
- The forward pass and loss calculation are repeated for 500 epochs.

Neural Network in Numpy

Backpropagation: gradients guide weight updates





Each point on the surface represents a setting of the weights; its height is the loss.
Backpropagation computes the slope $\frac{\partial L}{\partial w}$, and gradient descent takes a small step in the opposite,

Neural Network in Numpy

Backpropagation, step 1: start at the loss

Hochschule Karlsruhe University of Applied Sciences HKA

Training loop

```
learning_rate = 1e-6
for t in range(500):
  # Forward pass:  ...
  loss = np.square(y_pred - y).sum()
  # Backpropagation
    grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h_relu.T.dot(grad_y_pred)
  grad_h_relu = grad_y_pred.dot(w2.T)
  grad_h = grad_h_relu.copy()
  grad_h[h < 0] = 0
  grad_w1 = x.T.dot(grad_h)
  # Update weights
  w1 -= learning_rate * grad_w1
  w2 -= learning_rate * grad_w2
```

$$\frac{\partial L}{\partial y_{pred}} = 2(y_{pred} - y)$$

This gradient is the starting signal for backpropagation.

The chain rule starts with the local derivative of the loss with respect to its direct input.

Neural Network in Numpy

Backpropagation, step 2: gradient for w_2

Hochschule Karlsruhe University of Applied Sciences HKA

Training loop

```
learning_rate = 1e-6
for t in range(500):
  # Forward pass:  ...
  loss = np.square(y_pred - y).sum()
  # Backpropagation
  grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h_relu.T.dot(grad_y_pred)
  grad_h_relu = grad_y_pred.dot(w2.T)
  grad_h = grad_h_relu.copy()
  grad_h[h < 0] = 0
  grad_w1 = x.T.dot(grad_h)
  # Update weights
  w1 -= learning_rate * grad_w1
  w2 -= learning_rate * grad_w2
```

$$\underbrace{h_{relu}^T}_{H \times N} \times \underbrace{\frac{\partial L}{\partial y_{pred}}}_{N \times D_{out}} = \underbrace{\frac{\partial L}{\partial w_2}}_{H \times D_{out}}$$

The activation entering a layer is multiplied by the gradient leaving it.

This accumulates the contribution of every sample in the batch to each weight in w_2.

Neural Network in Numpy

Hochschule Karlsruhe University of Applied Sciences HKA

Backpropagation, step 3: pass the gradient to the hidden layer

Training loop

```
learning_rate = 1e-6
for t in range(500):
  # Forward pass:  ...
  loss = np.square(y_pred - y).sum()
  # Backpropagation
  grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h_relu.T.dot(grad_y_pred)
  grad_h_relu = grad_y_pred.dot(w2.T)
  grad_h = grad_h_relu.copy()
  grad_h[h < 0] = 0
  grad_w1 = x.T.dot(grad_h)
  # Update weights
  w1 -= learning_rate * grad_w1
  w2 -= learning_rate * grad_w2
```

$$\underbrace{\frac{\partial L}{\partial y_{pred}}}_{N \times D_{out}} \times \underbrace{w_2^T}_{D_{out} \times H} = \underbrace{\frac{\partial L}{\partial h_{relu}}}_{N \times H}$$

We multiply by w_2^T: the transpose reverses the forward operation $h_{relu} w_2$.

The result tells us how changing every hidden activation would change the loss.

Neural Network in Numpy

Backpropagation, step 4: back through ReLU

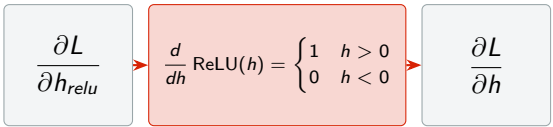
Hochschule Karlsruhe University of Applied Sciences HKA

Training loop

```
learning_rate = 1e-6
for t in range(500):
  # Forward pass:  ...
  loss = np.square(y_pred - y).sum()
  # Backpropagation
  grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h_relu.T.dot(grad_y_pred)
  grad_h_relu = grad_y_pred.dot(w2.T)
  grad_h = grad_h_relu.copy()

  grad_h[h < 0] = 0

  grad_w1 = x.T.dot(grad_h)
  # Update weights
  w1 -= learning_rate * grad_w1
  w2 -= learning_rate * grad_w2
```



The gradient is passed through where $h > 0$ and set to zero where ReLU was inactive.

In code, the mask is applied by setting `grad_h[h < 0]` to zero.

Neural Network in Numpy

Backpropagation, step 5: gradient for w_1

Hochschule Karlsruhe University of Applied Sciences HKA

Training loop

```
learning_rate = 1e-6
for t in range(500):
  # Forward pass:  ...
  loss = np.square(y_pred - y).sum()
  # Backpropagation
  grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h_relu.T.dot(grad_y_pred)
  grad_h_relu = grad_y_pred.dot(w2.T)
  grad_h = grad_h_relu.copy()
  grad_h[h < 0] = 0
  grad_w1 = x.T.dot(grad_h)
  # Update weights
  w1 -= learning_rate * grad_w1
  w2 -= learning_rate * grad_w2
```

$$\underbrace{x^T}_{D_{in} \times N} \times \underbrace{\frac{\partial L}{\partial h}}_{N \times H} = \underbrace{\frac{\partial L}{\partial w_1}}_{D_{in} \times H}$$

This is the same pattern as for w_2: input transpose times outgoing gradient.

We now have a gradient for every learnable weight matrix.

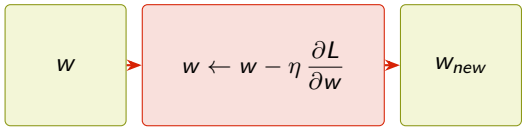
Neural Network in Numpy

Gradient descent: update the weights

Hochschule Karlsruhe University of Applied Sciences HKA

Training loop

```
learning_rate = 1e-6
for t in range(500):
  # Forward pass:  ...
  loss = np.square(y_pred - y).sum()
  # Backpropagation
  grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h_relu.T.dot(grad_y_pred)
  grad_h_relu = grad_y_pred.dot(w2.T)
  grad_h = grad_h_relu.copy()
  grad_h[h < 0] = 0
  grad_w1 = x.T.dot(grad_h)
  # Update weights
    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```



η (`learning_rate`) controls the size of the step.

Forward pass, loss, backpropagation, and update together make one training iteration.

Neural Network in Numpy

Hochschule Karlsruhe
University of
Applied Sciences
HKA

```
(ki-course) ➜  ~ python3 np.py
Epoch:0, loss:26311062.351246074
Epoch:50, loss:15323.53579119482
Epoch:100, loss:740.5489629434819
Epoch:150, loss:67.39831665785954
Epoch:200, loss:7.689127444872655
Epoch:250, loss:0.9645580491338863
Epoch:300, loss:0.1266611330191775
Epoch:350, loss:0.01704602978406973
Epoch:400, loss:0.0023282935849652363
Epoch:450, loss:0.00032123671398592195
```

The loss is decreasing, the network actually learns something.

Neural Network in Numpy

This was not so difficult, but also not very convenient:

- We had to implement the forward pass operations by hand.
- We had to derive the gradient computation for every layer of the network and compute it. (This will be very cumbersome for deep networks).
- And, most important, this only works on the CPU (and not on the GPU)!

Let's see how PyTorch can help us with that.

Neural Network in PyTorch


```python
import torch

dtype = torch.float
device = torch.device("cpu")
# device = torch.device("cuda:0") # Uncomment this to run on GPU

# N is batch size; D_in is input dimension;
# H is hidden dimension; D_out is output dimension.
N, D_in, H, D_out = 64, 1000, 100, 10

# Create random Tensors to hold input and outputs.
# Setting requires_grad=False indicates that we do not need to compute gradients
# with respect to these Tensors during the backward pass.
x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)
```

- We use PyTorch Tensors instead of Numpy arrays.
- Advantage: They can run automatically on the GPU!

Neural Network in PyTorch

Hochschule Karlsruhe University of Applied Sciences HKA

```
# Create random Tensors for weights.
# Setting requires_grad=True indicates that we want to compute gradients with
# respect to these Tensors during the backward pass.
w1 = torch.randn(D_in, H, device=device, dtype=dtype, requires_grad=True)
w2 = torch.randn(H, D_out, device=device, dtype=dtype, requires_grad=True)
```

- We also define the weights as PyTorch tensors.
- We can indicate that we want to have the gradients for these tensors automatically derived.
- Advantage: We can get rid of the part in which we defined the gradients for w_1 and w_2 ourselves.

Neural Network in PyTorch

Hochschule Karlsruhe
University of
Applied Sciences
HKA

```
learning_rate = 1e-6
for t in range(500):
    # Forward pass: compute predicted y using operations on Tensors; these
    # are exactly the same operations we used to compute the forward pass using
    # Tensors, but we do not need to keep references to intermediate values since
    # we are not implementing the backward pass by hand.
    y_pred = x.mm(w1).clamp(min=0).mm(w2)

    # Compute and print loss using operations on Tensors.
    # Now loss is a Tensor of shape (1,)
    # loss.item() gets the scalar value held in the loss.
    loss = (y_pred - y).pow(2).sum()
    if t % 100 == 99:
        print(t, loss.item())
```

- Forward pass and loss are analogous to NumPy but on tensors.

Neural Network in PyTorch


```
# Use autograd to compute the backward pass. This call will compute the
# gradient of loss with respect to all Tensors with requires_grad=True.
# After this call w1.grad and w2.grad will be Tensors holding the gradient
# of the loss with respect to w1 and w2 respectively.
loss.backward()
```

- PyTorch stores the differentiable operations that connect the input and weights to the loss in a computational graph:

$$x \to W_1 \to h \to \text{ReLU}(h) \to W_2 \to \hat{y} \to L$$

- When `loss.backward()` is called, it traverses this graph backwards and repeatedly applies the chain rule.
- More details in general and in PyTorch.

Neural Network in PyTorch


```
# Manually update weights using gradient descent. Wrap in torch.no_grad()
# because weights have requires_grad=True, but we don't need to track this
# in autograd.
# An alternative way is to operate on weight.data and weight.grad.data.
# Recall that tensor.data gives a tensor that shares the storage with
# tensor, but doesn't track history.
# You can also use torch.optim.SGD to achieve this.
with torch.no_grad():
    w1 -= learning_rate * w1.grad
    w2 -= learning_rate * w2.grad

    # Manually zero the gradients after updating weights
    w1.grad.zero_()
    w2.grad.zero_()
```

- Gradients are now available and can be used for gradient descent.

Neural Network in PyTorch

- This short example already showed two of the most powerful features of PyTorch:
  1. Operations can be run on the GPU without major code changes.
  2. Support for automatic calculation of gradients on tensor operations.
- On top of this, there is a more standardized way of defining neural networks, which packs everything in a standardized format.
- On the next few slides, we see the final code for this example using the `nn` package, which contains many standard Deep Learning modules.

Neural Network with `torch.nn`

Hochschule Karlsruhe
University of
Applied Sciences
HKA

```python
import torch


class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        """
        In the constructor we instantiate two nn.Linear modules and assign them as
        member variables.
        """
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        """
        In the forward function we accept a Tensor of input data and we must return
        a Tensor of output data. We can use Modules defined in the constructor as
        well as arbitrary operators on Tensors.
        """
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
```

Neural Network with `torch.nn`

Hochschule Karlsruhe
University of
Applied Sciences
HKA

```
# N is batch size; D_in is input dimension;
# H is hidden dimension; D_out is output dimension.
N, D_in, H, D_out = 64, 1000, 100, 10

# Create random Tensors to hold inputs and outputs
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)

# Construct our model by instantiating the class defined above
model = TwoLayerNet(D_in, H, D_out)
```

Neural Network with `torch.nn`

Hochschule Karlsruhe University of Applied Sciences HKA

```
# Construct our loss function and an Optimizer. The call to model.parameters()
# in the SGD constructor will contain the learnable parameters of the two
# nn.Linear modules which are members of the model.
criterion = torch.nn.MSELoss(reduction='sum')
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    # Forward pass: Compute predicted y by passing x to the model
    y_pred = model(x)

    # Compute and print loss
    loss = criterion(y_pred, y)
    if t % 100 == 99:
        print(t, loss.item())

    # Zero gradients, perform a backward pass, and update the weights.
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Working Environment

Hochschule Karlsruhe University of Applied Sciences HKA

- Your team can use the GPU hardware at HsKa-Lab. If you want to use that remotely there is a manual on GitHub. Not possible due to construction :(
- An alternative which previous courses also used: Google Colab
  - Google Log-in required (you can create a temporary one for this lab).
  - Cloud-based Jupyter notebook environment.
  - Free access to GPUs (but resources are limited).
- If you have GPUs on your machine, you can also install PyTorch locally.
- Another alternative to Colab is Kaggle Kernels.

Assignment 1

Hochschule Karlsruhe University of Applied Sciences HKA

Preparation:

1. Check out the notebook `0_Simple_NN.py` from GitHub, which describes how to set up a simple feedforward network on fake data. Try to understand everything and execute the code in a Jupyter notebook.
2. Read this blog post. It shows how to work with data loaders, how to load the MNIST dataset and how training is done in batches.
3. Read the famous blog post A Recipe for Training Neural Networks to learn more about tricks and best practices for model training.
4. Optional: Watch the official PyTorch Documentary.

Assignment 1

Hochschule Karlsruhe
University of
Applied Sciences
HKA

Task:

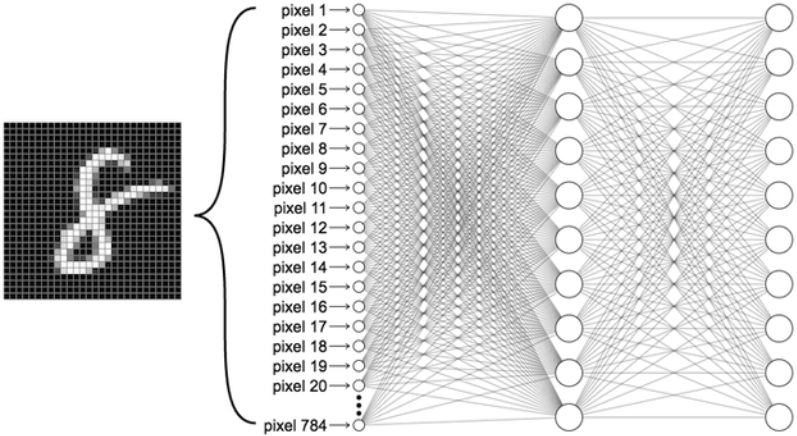
1. Load the MNIST dataset into train and test data loaders. Use the same parameters and transformations as in the blog post.
2. Create a feedforward neural network with an input layer, one hidden layer of size 100 and an output layer (same structure as in `0_Simple_NN.py`).
   - Adjust the input-layer size and flatten the MNIST tensor.
   - Use `log_softmax` as output activation function.
   - Do not use a CNN; use ReLU in the hidden layer.

Assignment 1

Task:

3. Train the network for 50 epochs using negative log likelihood loss. Create a plot of the training loss (without test loss).
4. Test the network on MNIST test data and report accuracy and loss.
5. Find out how to train on GPU instead of CPU. Compare training time; do not expect much improvement on this small data set.

Assignment 1



Output probabilities

$p(c = 0)$

$p(c = 1)$

$\vdots$

$p(c = 9)$

Image: https://ml4a.github.io/ml4a/looking_inside_neural_nets/

Submission Guidelines

Hochschule Karlsruhe University of Applied Sciences HKA

- Send your solutions as notebook file (`.ipynb`) to me via e-mail, mentioning your name in the e-mail.
- Use the following e-mail subject: `[KI-Lab] Assignment 1 - <teamname>`
- Deadline is next Thursday 11:59 p.m. (23:59).

Final words

Hochschule Karlsruhe University of Applied Sciences HKA

- You can start now working on assignment 1.
- It is up to you and your partners how you organize your time working on this.
- I will be around until the end of the slot at 1 pm for questions.
- Of course, questions can also be asked all time on Mattermost (but do not post any solution there!).

Any more questions now?

Happy Coding!